



CURSO TÉCNICO SUPERIOR PROFISSIONAL DE PROGRAMAÇÃO DE SISTEMAS DE INFORMAÇÃO
DESENVOLVIMENTO DE APLICAÇÕES

LINQ

LINQ

- Language-Integrated Query
 - Consultas integradas na linguagem
- É utilizado para consultar e atualizar dados
- Pode ser utilizado com classes `IEnumerable<>` para consultas em objetos em memória
- Suporta potencialmente qualquer tipo de conjunto de dados
 - Base de dados SQL, DataSet, XML, etc...
- Utilizado com `DbContext` \ `DbSet` (`IQueryable<>`) efetua a conversão para SQL para obter/gravar dados

OPERAÇÕES

Method Syntax	C# Query Expression Syntax
Select	select
SelectMany	Multiple from clauses.
Where	where
GroupBy	group ... by -or- group ... by ... into ...
GroupJoin<TOuter, TInner, TKey, TResult>(IEnumerable<TOuter>, IEnumerable<TInner>, Func<TOuter, TKey>, Func<TInner, TKey>, Func<TOuter, IEnumerable<TInner>, TResult>)	join ... in ... on ... equals ... into ...
Join<TOuter, TInner, TKey, TResult>(IEnumerable<TOuter>, IEnumerable<TInner>, Func<TOuter, TKey>, Func<TInner, TKey>, Func<TOuter, TInner, TResult>)	join ... in ... on ... equals ...
OrderBy<TSource, TKey>(IEnumerable<TSource>, Func<TSource, TKey>)	orderby
OrderByDescending<TSource, TKey>(IEnumerable<TSource>, Func<TSource, TKey>)	orderby ... descending
ThenBy<TSource, TKey>(IOrderedEnumerable<TSource>, Func<TSource, TKey>)	orderby ..., ...
ThenByDescending<TSource, TKey>(IOrderedEnumerable<TSource>, Func<TSource, TKey>)	orderby ..., ... descending

EXEMPLO

Query Syntax

```
using System;
using System.Linq;

class Program
{
    static void Main()
    {
        string[] words = {"hello", "wonderful", "LINQ", "beautiful", "world"};
        IEnumerable<String> shortWords = from word in words
                                         where word.Length <= 5
                                         select word;

        foreach (String word in shortWords)
        {
            Console.WriteLine(word);
        }

        Console.ReadLine();
    }
}
```

Method Syntax

```
using System;
using System.Linq;

class Program
{
    static void Main()
    {
        string[] words = {"hello", "wonderful", "LINQ", "beautiful", "world"};
        IEnumerable<String> shortWords = words
                                         .where(w => w.Length <= 5);

        foreach (String word in shortWords)
        {
            Console.WriteLine(word);
        }

        Console.ReadLine();
    }
}
```

Output: hello LINQ world

INTERPRETAR UM METODO EM LINQ

`OrderBy<TSource, TKey>(IEnumerable<TSource>, Func<TSource, TKey>)`

OrderBy<TSource, TKey> / Definição dos tipos genéricos, Podem ser definidos ou inferidos pelos elementos usados

(IEnumerable<TSource>, / Coleção Externa (this)

Func<TSource, TKey>) / Função para obter a chave de ordenação

IEnumerable<TSource>

Func<TSource, TKey>

`var orderedCustomers = customers.OrderBy(c => c.Name);`

c – cada elemento dos customers (TSource)

c.Name – atributo pelo qual será ordenado (TKey)

Func<T1, T2, ..., TResult>

T1, T2, ... - Tipos dos parâmetros de entrada da função

TResult - Tipo do retorno da função.

OPERADORES STANDARD

Classification	Standard Query Operators
<i>Filtering</i>	Where, OfType
<i>Sorting</i>	OrderBy, OrderByDescending, ThenBy, ThenByDescending, Reverse
<i>Grouping</i>	GroupBy, ToLookup
<i>Join</i>	GroupJoin, Join
<i>Projection</i>	Select, SelectMany
<i>Aggregation</i>	Aggregate, Average, Count, LongCount, Max, Min, Sum
<i>Quantifiers</i>	All, Any, Contains
<i>Elements</i>	ElementAt, ElementAtOrDefault, First, FirstOrDefault, Last, LastOrDefault, Single, SingleOrDefault
<i>Set</i>	Distinct, Except, Intersect, Union
<i>Partitioning</i>	Skip, SkipWhile, Take, TakeWhile
<i>Concatenation</i>	Concat
<i>Equality</i>	SequenceEqual
<i>Generation</i>	DefaultEmpty, Empty, Range, Repeat
<i>Conversion</i>	AsEnumerable, AsQueryable, Cast, ToArray, ToDictionary, ToList

OPERAÇÕES BÁSICAS – QUERY SYNTAX

```
public class Customer
{
    public string Name { get; set; }
    public string City { get; set; }
}
```



```
var customers = new List<Customer>
{
    new Customer { Name = "Ana", City = "Lisbon" },
    new Customer { Name = "João", City = "Porto" },
    new Customer { Name = "Maria", City = "London" },
    new Customer { Name = "Pedro", City = "London" }
};
```

- **Obter dados: select**

// queryAllCustomers é um IEnumerable<Customer>

```
var queryAllCustomers = from cust in customers
                        select cust;
```

- **Aplicar filtros: where**

```
var queryAllCustomers = from cust in customers
                        where cust.City == "London"
                        select cust;
```

Podem ser combinadas várias condições na clausula **where** utilizando os operadores **&&** ou **|**



OPERAÇÕES BÁSICAS – METHOD SYNTAX

- **Obter dados: select**

```
// queryAllCustomers é um IEnumerable<Customer>
```

```
var queryAllCustomers = customers  
    .select(cust => cust);
```

- **Aplicar filtros: where**

```
var queryAllCustomers = customers  
    .where(cust => cust.City == "London")  
    .select(cust => cust);
```

Podem ser combinadas várias condições na clausula *where* utilizando os operadores **&&** ou **||**



OPERAÇÕES BÁSICAS – APLICADO A DBCONTEXT

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string City { get; set; }
}
```



```
public class AppDbContext : DbContext
{
    public DbSet<Customer> Customers { get; set; }
}
```

```
using (var context = new AppDbContext())
{
    var clientesLondres = context.Customers
        .Where(w => w.City == "London")
        .OrderBy(c => w.Name)
        .ToList();

    foreach (var cust in clientesLondres)
    {
        Console.WriteLine($"{cust.Name} - {cust.City}");
    }
}
```

ToList – Cria código SQL, executa e devolve uma lista de objetos para memória. Até o ToList apenas estamos a criar a query.



```
SELECT Id, Name, City
FROM Customer
WHERE City='London'
ORDER BY Name
```

OPERAÇÕES BÁSICAS – ORDERBY

■ Ordenar: orderby

```
var queryAllCustomers =  
from cust in customers  
where cust.City == "London"  
orderby cust.Name ascending  
select cust;
```

```
var queryAllCustomers =  
customers  
.where(cust => cust.City == "London")  
.orderby(cust => cust.Name)  
.ToList();
```

```
SELECT Id, Name, City  
FROM Customer  
WHERE City='London'  
ORDER BY Name
```

```
queryAllCustomers » IEnumerable<Customer>
```

OPERAÇÕES BÁSICAS – GROUPBY

■ Agrupar: group ... by ...

```
// queryCustomersByCity é um IEnumerable<IGrouping<string, Customer>>
var queryCustomersByCity =
    from cust in customers
    group cust by cust.City;

// customerGroup é um IGrouping<string, Customer>
foreach (var customerGroup in queryCustomersByCity) {
    Console.WriteLine(customerGroup.Key);
    foreach (Customer customer in customerGroup) {
        Console.WriteLine(" {0}", customer.Name);
    }
}
```

City

IEnumerable<Customer>

OPERAÇÕES BÁSICAS – GROUPBY INTO

■ Agrupar: group ... by ... into

```
// custQuery é um IEnumerable<IGrouping<string, Customer>>  
var custQuery =  
    from cust in customers  
    group cust by cust.City into custGroup  
    where custGroup.Count() > 2  
    orderby custGroup.Key  
    select custGroup;
```

Permite que se executem outras ações sobre o grupo criado

Em Method Syntax não existe um “into” explícito, este é feito através da chamada a métodos após o GroupBy:

```
var result = customers.GroupBy(cust => cust.City).Where(g => g.Count() > 5).OrderBy(g => g.Key).ToList();
```

OPERAÇÕES BÁSICAS – JOIN / PROJECTIONS

■ Joining with projections (selecting)

```
var innerJoinQuery =  
    from cust in customers  
    join dist in distributors  
        on cust.City equals dist.City  
    select new { CustomerName = cust.Name,  
                DistributorName = dist.Name };
```

```
SELECT c.Name AS CustomerName, d.Name AS DistributorName  
FROM Customers AS c  
INNER JOIN Distributors AS d ON c.City = d.City;
```

```
foreach (var item in innerJoinQuery)  
{  
    Console.WriteLine($"Customer: {item.CustomerName}, Distributor: {item.DistributorName}");  
}
```

SINTAXE MISTA

- Ambas as sintaxes podem ser interligadas, já que os objetos não distinguem entre sintaxe
- As seguintes linhas de código têm o mesmo resultado

```
var queryAllCustomers = from cust in customers
    where cust.City == "London"
    select cust;
int customerCount = queryAllCustomers.Count();
```

ou

```
int customerCount = (from cust in customers
    where cust.City == "London"
    select cust).Count();
```

FONTES

<https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/>

<https://www.tutorialsteacher.com/linq/linq-tutorials>